Looking at this code, the vulnerabilities are fairly clear once you go through it line by line.

The most obvious one is that passwords are stored as plain text on the User object, so anyone with database or memory access gets the actual password straight away, no cracking required. Shah (2023) makes the point that plaintext credential storage remains one of the most common and avoidable failures in application security, and that hashing with a modern algorithm such as bcrypt or argon2 should be treated as a baseline requirement rather than an enhancement. There's also no input validation at all on add_user or authenticate, so empty strings or unexpected input just get accepted without question. On top of that there's no password policy, which is why "admin123" and "password" are both allowed through without complaint, something Baker (2022) discusses in relation to authentication design in Python and Django applications, where enforcing complexity rules at the point of registration is treated as a core defensive layer. There's also nothing stopping someone hammering authenticate() repeatedly, so brute forcing is wide open.

One thing I'd flag slightly differently to the task description though, calling the malicious_input example "SQL injection" is a bit misleading, since there's no actual SQL or database query happening here at all, it's just a plain Python string comparison. Rehim and Mohan (2024) note that injection style attacks depend specifically on unsanitised input reaching a query or command interpreter, which isn't happening in this snippet, so the real underlying issue here is closer to a lack of input validation generally rather than SQL injection specifically.

I've refactored the code separately to address hashing, validation and basic rate limiting, and I'll share that in a follow up post to keep this one within the word limit.

References

Baker, M. (2022) *Secure web application development: a hands on guide with Python and Django*. Berkeley, CA: Apress.

Rehim, R. and Mohan, M. (2024) *Offensive security using Python: a hands on guide to offensive tactics and threat mitigation using practical strategies*. Birmingham: Packt Publishing.

Shah, M. (2023) *Cloud native software security handbook: unleash the power of cloud native tools for robust security in modern applications*. Birmingham: Packt Publishing.